

Sprawozdanie

Projekt Sieci Komputerowe 2026

Natalia Sitkowska

Mateusz Ożana

Podział obowiązków:

- Natalia Sitkowska - poprawki otrzymanego kodu, dalsza modyfikacja ostatecznego protokołu sieciowego, optymalizacja kodu
- Mateusz Ożana - zmodyfikowanie aplikacji aby miała ona funkcjonalność sieciową oraz zaprogramowanie pierwszych wersji protokołu komunikacyjnego oraz serializacji danych.

(Większość obowiązków nie miała konkretnego podziału)

Opis działania protokołu komunikacyjnego i serializacja:

Aplikacja rozdziela symulacje na dwie instancje. **Obiekt** będący serwerem i nasłuchujący na wybranym porcie. **Regulator** będący klientem i łączący się z adresem IP obiektu.

Wszystkie przesyłane dane serializowane są binarnie za pomocą QDataStream. Każdy pakiet ma operatory << (serializacja) oraz >> (deserializacja) oraz zawiera kolejno swoją długość, typ oraz unikalne dane.

Powstały trzy typy przesyłanych pakietów:

- ControlPacket (Regulator -> obiekt) przesyła numer pakietu, sterowanie, oraz wartość zadaną
- OutputPacket (Obiekt -> regulator) przesyła numer pakietu oraz wartość regulowaną
- ConfigPacket (W obie strony) w momencie zmiany jakiegokolwiek wartości modyfikowalnej w oknie symulacji przez użytkownika, przesyła całą uaktualnioną konfigurację do drugiej instancji.

Wykrywanie gubienia pakietów:

Każdy pakiet który reprezentuje to co widzimy na wykresach posiada swój sekwencyjny numer. Jeżeli regulator wyśle aktualne ControlPacket do obiektu, a odpowiedź zawierająca ten sam numer pakietu nie wróci do czasu wzbudzenia zegara i wygenerowania kolejnej próbki, otrzymana spóźniona wartość jest całkowicie odrzucana. W takim przypadku:

- rysowana jest najnowsza zaakceptowana wartość od obiektu,
- regulator kontynuuje na najnowszej zaakceptowanej wartości,
- licznik zgubionych pakietów rośnie. Jeżeli licznik dojdzie do 5-ciu pod rząd to wtedy następuje zerwanie połączenia i symulacja kontynuowana jest w trybie lokalnym na najnowszych danych.

Historia rozwoju aplikacji o funkcjonalności sieciowe:

1. **Usunięto błędy** celowo wprowadzone przez twórców kodu, zastosowano poprawki innych niedociągnięć. Pomimo że ten punkt znajduje się na początku to w trakcie całego modyfikowania kodu napotykanne były pojedyncze ubytki w funkcjonalności, które w jakimś stopniu spowalniały prace.
2. **Pierwsze sieciowe przesłanie** wartości testowych, połączenie wykonano poprzez dane "na sztywno" wpisane w kodzie oraz sterowano połączeniem za pomocą konsoli. Pierwsza wersja ramki sieciowej wraz z serializacją i deserializacją
3. Stworzono odmienny **wygląd trybu sieciowego** dla obiektu oraz regulatora
4. Stworzono **fundamenty pod przesył sieciowy** wszelkich modyfikowalnych wartości w oknie symulacji, oraz dodano pierwszą wersję przesyłu tych danych
5. **Modyfikacja przesyłanych pakietów**, pierwsza wersja łączyła błędnie ze sobą odmienne dane, dopiero tutaj powstały trzy ostateczne pakiety
6. Dodano **funkcjonalność ignorowania spóźnionych pakietów** – zauważenie poważnego problemu z wydajnością
7. **Optymalizacja rysowania wykresów** – spowodowało poprawę ale wciąż przesył informacji w oczekiwanym interwale był niemożliwy
8. **Dodano symulacje w tle** w taki sposób, aby obie instancje wykonywały własne pełne obliczenia. Dzięki temu w przypadku zerwania połączenia obie instancje mogą kontynuować lokalnie
9. **Osobny timer do wykresów** – ostateczne rozwiązanie problemu z wydajnością przesyłania pakietów

Najistotniejsze napotkane trudności:

- **Znalezienie oraz naprawa błędów pozostawionych po poprzedniej grupie.**

Ponieważ w kodzie znajdowało się kilka mocno zauważalnych na początku problemów, ciężko było nam stwierdzić czy aplikacja kiedykolwiek działała tak jak powinna i czy wyeliminowaliśmy już błędy wprowadzone celowo. Dużym szokiem było jak bardzo niektóre rozwiązania różnią

się od tych które my zastosowaliśmy we własnym kodzie i to utrudniało jego analizę. Aby znaleźć drugi celowo wprowadzony błąd zwróciliśmy się do jego twórców.

- **Mała wydajność kodu** (jedyne istotne problemy w kwestii komunikacji sieciowej)

Po modyfikacji kodu tak aby nastąpiło rozłączenie po 5 zgubionych pakietach, zauważyliśmy że symulacja poważnie się nie wyrabia. Komunikacja sieciowa ledwo działała na interwale sięgającym około 400ms. Pierwszą poprawką była modyfikacja ówczesnych metod.

Zauważyliśmy powtarzające się czynności które można było zoptymalizować, szczególnie sam sposób rysowania wykresów. To pomogło nieznacznie. Tym co ostatecznie rozwiązało problem było stworzenie osobnego timera dla rysowania wykresów, w ten sposób nawet jeżeli interwał symulacji wynosi <50ms, wykresy zapamiętują wszystkie otrzymane próbki ale aktualizują je dopiero co 100ms.

Czego się nauczyliśmy:

Mateusz Ożana:

- Pracy z projektem przejętym po innej grupie ludzi
- Działania komunikacji sieciowej w C++
- Dostosowywania projektów wcześniej nie przystosowanych do komunikacji sieciowej

Natalia Sitkowska:

- Praca na obcym kodzie dodaje całkowicie nowe wcześniej nie napotkane problemy, zamiast z pewnością podważać swoją własną pracę i wprowadzone funkcjonalności trzeba liczyć się z tym że błędy mogły zostać popełnione już dawno przed naszym udziałem w kodzie
- Wiele aspektów komunikacji sieciowej które nie były mi wcześniej kompletnie znane
- Praktyczny przykład o tym jak ważna jest wydajność kodu
- Wykorzystanie nowszych modeli AI do optymalizacji pracy na kodzie